

GreenStep Challenge Admin Console

Project Report

Team 2

Josh Dunlap Eli Goldberger Khue Vo Rudy Vergara

CISC 480: Senior Capstone, Section 01

University of St. Thomas

Community Partner: Minnesota Pollution Control Agency (MPCA)

May 22, 2026

Document Type: Project Report

Project: GreenStep Challenge Admin Console

Repository: https://github.com/CISC-480-GreenStep/sp26_team2_greenstepchallenge_admin

Live URL: <https://greenstep.auriga.fyi>

Version: v0.9.0 (final capstone build)

Submission Date: May 22, 2026

Contents

1	Project Overview	3
1.1	Problem Statement	3
1.2	Purpose	3
1.3	Target Users (User Profile)	3
1.4	Constraints	3
1.5	Proposed Solution	4
1.6	Technology Stack	4
1.7	Expected Benefits	4
2	Current Functionality & Features	5
2.1	Authentication and Authorization	5
2.2	User Management	5
2.3	Group Management	5
2.4	Challenge Management	5
2.5	Challenge Templates	6
2.6	Customizable Dashboard	6
2.7	Reports and Comparison Mode	6
2.8	Documentation	6
3	To-Do List & Future Enhancements	7
3.1	Technical	7
3.2	Functional	7
3.3	Usability	7
3.4	Future Expansion	8
3.5	Known Bugs and Limitations	8
	References	9

1 Project Overview

1.1 Problem Statement

The Minnesota Pollution Control Agency (MPCA) runs annual sustainability challenges that encourage staff to log everyday actions such as biking to work, eliminating single-use plastics, and reducing food waste. Before this project, the agency’s Sustainability Program Coordinator, Kristin Mroz-Risse, administered each challenge through a sprawling network of spreadsheets, manual email reminders, and ad hoc reporting. The spreadsheet workflow scaled poorly across multiple concurrent challenges (department-level cohorts, agency-wide pushes, Earth Month campaigns), made it difficult to answer the basic question “*who is actually engaged?*”, and left no audit trail when participants joined or left a group.

1.2 Purpose

The GreenStep Challenge Admin Console replaces the spreadsheet workflow with a single web application that lets MPCA staff:

- design challenges and reusable templates,
- organize participants into flexible groups (departments, cohorts, building-level competitions),
- monitor engagement on a customizable dashboard, and
- export participation reports for leadership.

The application is the back office for the broader GreenStep program; a sibling mobile application built by Team 1 in the same semester gives the participants themselves a way to log completed actions against a shared database.

1.3 Target Users (User Profile)

The application exposes three role tiers, enforced both in the React UI and at the database layer via Row-Level Security:

- **SuperAdmin** – Kristin and her direct delegates. Full control: invite new admins, deactivate or permanently delete users, manage every group / challenge / template.
- **Admin** – department-level coordinators. Can create and manage challenges, groups, and templates; cannot create or delete top-level users.
- **GeneralUser** – the rest of MPCA staff. Read-only inside the admin app; participation itself happens in the sibling mobile application.

1.4 Constraints

- **Project duration:** ten weeks of active development by a four-person student team.
- **Hosting budget:** zero. The deployment had to fit on free tiers (Vercel hobby, Supabase free).
- **Network environment:** MPCA staff inboxes are protected by corporate link-scanning, which forced the team to abandon magic-link authentication in favor of an 8-digit one-time code.
- **Shared database:** the schema is co-owned with Team 1, requiring coordination on table and column shape.
- **Browser support:** modern evergreen browsers only; no Internet Explorer 11 support.

1.5 Proposed Solution

A single-page React 19 application hosted on Vercel, talking directly to a managed Supabase project (Postgres + Auth + REST + Row-Level Security). The design intentionally has no custom backend tier; authorization is enforced inside the database through RLS policies that read the role claim from the caller’s JWT. Two narrow Vercel serverless functions exist only for operations the anon key cannot perform (inviting a new user, permanently deleting a user account).

This shape gave the team three benefits relative to the original plan of a separate FastAPI service:

1. one fewer container to deploy, monitor, and secure;
2. authorization that cannot be bypassed from the browser console;
3. the shared database becomes a literal contract between Team 2 (the admin app) and Team 1 (the mobile app).

1.6 Technology Stack

Layer	Technology
Frontend	React 19 (JavaScript)
Build tool	Vite
UI library	Material UI (MUI) v7 with a custom dark theme
Charts	Recharts
Dashboard grid	<code>react-grid-layout</code> v2 (drag-and-drop)
Routing	React Router v7
Database	Postgres 15 (via Supabase)
Auth	Supabase Auth (8-digit OTP code over email)
Authorization	Postgres Row-Level Security with role-based policies
Privileged ops	Vercel Serverless Functions (Node.js) with Supabase service-role key
Hosting	Vercel (SPA + functions); Supabase (database + auth)
Quality gates	ESLint flat config, Prettier, file-size cap (300 lines / file)

1.7 Expected Benefits

- **For Kristin and MPCA:** a single tool that replaces the spreadsheet workflow; cross-challenge reports in seconds; clear answer to “who is engaged?”
- **For challenge participants (indirectly):** cleaner program operations, fewer broken emails, faster reminders.
- **For the next student team:** a documented codebase with every file under 300 lines, a consolidated baseline schema, and the C4 architecture, ER diagram, and sequence diagrams already on paper.

2 Current Functionality & Features

The capstone build (v0.9.0 plus the May 22, 2026 follow-on fixes merged in PRs #79–#87) ships the following features. Every item in this list is implemented on the live deploy at <https://greenstep.auriga.fyi>.

2.1 Authentication and Authorization

- Passwordless sign-in using an 8-digit one-time code emailed through Supabase Auth. Pre-flight check rejects unregistered or deactivated emails before sending the code.
- `RequireAuth` route guard with optional `minRole` prop; loading spinner during session restore prevents the “flashed login page” effect on hard refresh.
- Row-Level Security enforces the same role checks at the database layer, so even a malicious caller using the anon key directly cannot mutate admin data.
- Dev shortcut: a “Quick Login as Kristin (SuperAdmin)” button on the login page uses `signInWithPassword` against a seeded dev account. Will be removed before any real production hand-off.

2.2 User Management

- Search and filter the user list by role and group; CSV export.
- Per-user detail page with activity log, participation history, total points, and per-challenge progress bars.
- Inline *Change Group* dropdown for Admins / SuperAdmins.
- Activate / Deactivate flow with a `ConfirmDialog`; deactivation soft-removes by flipping a status flag and revoking the auth credential, preserving the audit trail.
- SuperAdmin-only *Permanent Delete* (issue #67) wired through a Vercel API route using the Supabase service-role key; cascades `participation` and `activity_logs` rows while preserving anything the user authored (`createdBy` becomes NULL).

2.3 Group Management

- Full CRUD for flexible groups (cities, departments, cohorts).
- Group detail page with member list, member add/remove, challenges scoped to the group, and bidirectional links to both.
- Member and challenge counts on the group list are clickable and navigate to a filtered view.

2.4 Challenge Management

- List view with search, status filter, group filter, and CSV export.
- Create and edit form with a four-color theme picker (header background, header text, body background, body text) and a live mobile preview rendered alongside the form on **md+** screens.
- *Quick Start* panel that lets the admin seed a new challenge from either a saved **Template** or a previously **Archived Challenge** (PR #72).
- Challenge detail view: **Participants** table, full participation log, clickable group link, archive button, delete button.
- Action CRUD per challenge: add, edit, and delete the actions associated with a single challenge (name, description, category, points).

2.5 Challenge Templates

- Reusable challenge templates with pre-configured actions.
- Three seed templates: H2O Hero Week, Power Down Challenge, Sustainable Commute Week.
- Applying a template when creating a new challenge pre-fills form fields and stages template actions to attach on submit.

2.6 Customizable Dashboard

- 23 available widgets organized into four categories: *Overview Stats*, *Charts & Visualizations*, *Tables & Lists*, and *Reports & Analysis*.
- Drag-and-drop reorder and corner-resize via `react-grid-layout`, gated behind an explicit *Customize* edit mode.
- Four *Quick Layout Presets*: Default, Executive Summary, Analytics Deep Dive, Compact Overview.
- Layout persists per user to `localStorage` and survives page refreshes.
- HCI-informed affordances: grip handles and dashed borders only appear in edit mode (progressive disclosure); a snackbar confirms saves (feedback); the user cannot remove every widget (error prevention).

2.7 Reports and Comparison Mode

- Standalone `/reports` page with filter by challenge and date range, a category-breakdown chart, the full participation table, and CSV export.
- New *Participation Report* widget on the dashboard (PR #68) that mirrors the same filterable table + CSV export inside a draggable, resizable widget. Each widget instance keeps its own filter state so multiple copies can be dropped on the grid for side-by-side comparisons.
- *Comparison Mode* (PR #72) promoted from a nested dashboard menu to a dedicated toolbar button, with an integrated challenge picker.

2.8 Documentation

- Top-level `README.md` (project overview, setup, version history) and `CODING_GUIDELINES.md` (Airbnb-style + project-specific conventions).
- `docs/ARCHITECTURE.md` – the contributor’s map.
- `docs/product-documentation/PRODUCT_DOCUMENTATION.pdf` – the capstone Product Documentation deliverable: PRD, UX/UI spec, Technical Design, Source Code Documentation, and QA Activities Document, all in one 24-page LaTeX-built PDF.
- `retrospectives.md` – three documented retrospectives (4Ls, Sailboat, Mad/Sad/Glad).
- `supabase/migrations/` – consolidated baseline: `001_initial_schema.sql` (tables, indexes, RPCs, RLS, permissions) and `002_seed_data.sql`.

3 To-Do List & Future Enhancements

The team intentionally hands this section off to the spring 2027 student team. Items are grouped by category and tagged with the GitHub issue or PR that tracks them where one exists.

3.1 Technical

- **Drop legacy presets / preset.actions tables** (issue #88). The rename from “Preset” to “Template” was completed at the UI and API level in v0.9.0, but the legacy tables were never dropped. One line of code in `data/api/templates.js:92` (`deleteTemplate`) still queries the legacy table and silently no-ops.
- **Reconcile challenges.theme (legacy single color) with the four bgColor*/txColor* columns added in migration 014** (issue #89). Dashboard widgets read the legacy column; the form writes only the four split columns. Either migrate widgets or back-fill the legacy column on insert.
- **Sync the dashboard layout to the server** so it follows the user across devices, replacing the current `localStorage`-only persistence.
- **Tighten the action → category foreign key.** The `actions.category` and `challenges.category` columns store the category name as text rather than a foreign key to the new `categories` table. FK normalization is a clean future migration.

3.2 Functional

- **Finish issue #40 (Combine Dashboard and Report Engine).** Slice 1 (the Participation Report widget) landed in PR #68. Slices 2 and 3 – renaming the sidebar entry from “Dashboard” to “Report and Analysis,” adding a *Reporting* quick-layout preset, and retiring the standalone `/reports` route – are scoped but not started.
- **Implement Badges and Tickets pages** (issue #90). Draft pages exist on the closed PR #80 branch but the underlying `badges` and `tickets` tables return 404 on the live database. Both schema and UI finishing work are needed; the PR also stubbed out a leaderboard query that must be reverted.
- **Photo moderation flow.** Participation rows already have a `photoUrl` column; the actual upload, review, and moderation UI was out of scope for this semester.
- **Push notifications** for challenge reminders. The mobile app (Team 1) is the natural surface; the admin app would only need to enqueue the messages.
- **Saved report templates.** The standalone Reports page filters reset on navigate-away. Persisting named filters would let admins jump straight to their saved cross-challenge slice.

3.3 Usability

- **Keyboard-accessible dashboard reorder.** Mouse drag works; keyboard-only users currently fall back to the Quick Layout Presets.
- **Theme presets for the challenge color picker.** The four color pickers default to a fixed palette; an admin-curated swatch of MPCA brand colors would speed up authoring.

- **Inline validation on the challenge form** for the end-date-must-be-after-start-date constraint. Currently the database constraint is the source of truth; the form surfaces the resulting error message but does not pre-validate.

3.4 Future Expansion

- **Full integration with the Team 1 mobile app.** The shared schema is in place but the two apps have not been exercised together end-to-end at any scale.
- **Multi-tenant support.** If MPCA's program is ever adopted by other state agencies, the schema would need a tenant identifier on every table or a separate Supabase project per tenant.
- **Advanced analytics.** Trend lines over multiple years, cohort comparisons, predictive engagement indicators – all out of scope for the spring 2026 capstone but natural follow-ons once two or three years of participation data accumulate.
- **Internationalization.** The current UI is English-only.

3.5 Known Bugs and Limitations

- Repository migration files were drifted from the live deployed schema for most of the semester. Reconciled and squashed to a two-file baseline in PRs #85 and #86. Documented so the next team doesn't fall into the same trap.
- One template apply path still references the legacy `theme` text field rather than the four split colors; folded into issue #89.

References

- [1] Minnesota Pollution Control Agency, “GreenStep Cities Sustainability Program.” [Online]. Available: <https://www.mnpca.state.mn.us/>. [Accessed: May 21, 2026].
- [2] S. Brown, *The C4 model for visualising software architecture*. [Online]. Available: <https://c4model.com/>. [Accessed: May 21, 2026].
- [3] Supabase, “Row Level Security.” [Online]. Available: <https://supabase.com/docs/guides/auth/row-level-security>. [Accessed: May 21, 2026].
- [4] Supabase, “Database Migrations.” [Online]. Available: <https://supabase.com/docs/guides/deployment/database-migrations>. [Accessed: May 21, 2026].
- [5] Material UI, “Material UI v7 Documentation.” [Online]. Available: <https://mui.com/material-ui/>. [Accessed: May 21, 2026].
- [6] Vercel, “Functions Overview.” [Online]. Available: <https://vercel.com/docs/functions>. [Accessed: May 21, 2026].
- [7] J. Patton, *User Story Mapping: Discover the Whole Story, Build the Right Product*. O’Reilly Media, 2014.
- [8] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1.” [Online]. Available: <https://www.w3.org/TR/WCAG21/>. [Accessed: May 21, 2026].